

Ejercicio 1 - Overloading y relaciones entre clases

2027 FIFA Women's World Cup

Objetivo

- Diferenciar entre definición y creación de objetos.
- Modelar su programa en UML.
- Aplicar de forma correcta los modificadores de visibilidad para no violar el encapsulamiento.
- Creación de un programa principal que resuelva el problema.

Introducción

El mundial 2027 femenino de la FIFA planea llegar a su ciudad y están solicitando ayuda de programadores locales para hacer un experimento en las ventas de tickets. Debido a la inconformidad de los fans en los eventos anteriores por lo rápido que se vendieron los boletos se decidió implementar un nuevo algoritmo de compra aleatorio que sea justo y de las mismas posibilidades a todos.

Para poder comprar los boletos se debe hacer lo siguiente:

1. Solicitar una compra de un boleto. Para hacer esto se debe ingresar el nombre, email, cantidad de boletos a comprar y presupuesto máximo.
2. Al hacer la solicitud de compra se genera un número de ticket aleatorio de 1 a 15,000.
3. Para determinar si un ticket puede comprar boletos se debe generar 2 números aleatorios (a y b) adicionales dentro del mismo rango 1 a 15,000. Se debe validar que el ticket esté dentro del rango generado por los 2 números adicionales (a y b). Si esto se cumple, el ticket es apto para comprar boletos.
4. Debido a que este proceso es más complejo, solo se usará como piloto para vender 60 entradas, distribuidas de forma igualitaria en 3 localidades diferentes:
 - a. Localidad 1 - La más alejada del escenario pero la más barata con un precio de \$100.
 - b. Localidad 5 - En medio y con una mejor vista al escenario pero con un precio de \$500.
 - c. Localidad 10 - Hasta adelante, la mejor vista del concierto pero la más cara con un precio de \$1000.
5. Cuando un ticket es apto para compra se debe seleccionar de forma aleatoria la localidad a asignarle dentro de las 3 disponibles.
6. Al tener la localidad definida se deben realizar 3 validaciones:
 - a. Validar espacio: no es posible vender los tickets si la localidad no tiene espacio, es decir si ya se vendieron los 20 espacios disponibles.
 - b. Validar disponibilidad de los boletos deseados: se debe validar que exista espacio para la cantidad de boletos que el comprador quiere comprar. Si no es así se le

- vende la mayor cantidad posible sin exceder el límite deseado.
- c. Validar precio: si el precio de la sección es mayor al presupuesto no se puede comprar el boleto y se termina el proceso para ese comprador.
7. Si pasan las validaciones anteriores, se le vende al usuario la cantidad de boletos determinada.

Ejercicio

Análisis

Cree una tabla en la que se incluya toda la información solicitada. Puede hacer una tabla para propiedades y otra para métodos:

1. ¿Qué propiedades y métodos tendrá cada clase?
2. ¿Qué tipo de datos deben tener las propiedades y métodos de cada clase?
3. ¿Cuáles deben ser los modificadores de visibilidad de los miembros en cada clase?
4. ¿Qué parámetros serán requeridos por los métodos en sus clases?
5. ¿Qué propósito tendrá dentro de la clase?

Responda las siguientes preguntas:

6. ¿Cómo clasifica cada una de sus clases según el patrón MVC (Model, View, Controller)? Justifique brevemente el rol que cumple cada una dentro de esa clasificación.
7. ¿Qué ventaja ofrece separar la lógica de negocio (Model) de la interacción con el usuario (View/Controller) en este sistema? ¿Qué problema evita esta separación si en el futuro se necesita modificar el algoritmo de asignación de tickets?

Diseño

Desarrolle un diagrama de clases que muestre los miembros de cada clase y que establezca una relación entre ellas. No se aceptará el uso de asociación genérica entre las clases: debe determinar y representar explícitamente si la relación es de agregación o composición, según corresponda. Utilice un color diferente para cada tipo de clase según su rol dentro del patrón MVC (Model, View, Controller) y tome en cuenta esta clasificación al definir las relaciones entre clases. Incluya en su diagrama al driver program (que contiene el método `main()`). Use anotaciones (recuadros de texto) en su diagrama para aclarar cualquier información que considere necesaria.

Programa

Desarrolle un sistema que contenga un driver program con instancias de: localidad y comprador como mínimo. El sistema debe estructurarse respetando la separación de responsabilidades del patrón MVC definida en su Diseño. Así mismo debe mostrar un menú con las siguientes opciones:

1. Nuevo comprador: reemplaza la instancia actual del comprador activo.
2. Nueva solicitud de boletos: permite al comprador actual participar para comprar boletos.
3. Consultar disponibilidad total: se muestran cuántos boletos se han vendido en cada localidad y cuantos boletos hay disponibles para las 3 localidades.
4. Consultar disponibilidad individual: se le pide al usuario que defina una localidad y se

muestran solo los boletos disponibles para la localidad seleccionada.

5. Reporte de caja: mostrar cuánto se ha generado de dinero dados los boletos vendidos en las 3 localidades.
6. Salir: termina la ejecución del programa.

Material a entregar en Canvas

- Archivo PDF con el análisis y diseño.
- Captura de pantalla del merge request a su branch en el repo de la clase.

Evaluación

- **Análisis** (30 puntos)
 - **(10 puntos)** Tabla de atributos de todas las clases.
 - **(10 puntos)** Tabla de métodos de todas las clases.
 - **(10 puntos)** Preguntas.
- **Diseño** (30 puntos)
 - **(15 puntos)** Están representados cada uno de los componentes de la clase de manera correcta, clasificados por color según su rol en MVC, y coinciden con lo descrito en el análisis y con el estándar de UML.
 - **(15 puntos)** Las relaciones entre las clases están correctas, utilizando agregación o composición según corresponda (no se acepta asociación genérica).
- **Programa** (40 puntos)
 - **(10 puntos)** Las 6 opciones del menú están implementadas y funcionan según lo descrito en el enunciado.
 - **(10 puntos)** El algoritmo de asignación está correctamente implementado: generación del ticket, generación de los números a y b, validación de rango, y selección aleatoria de localidad.
 - **(10 puntos)** Las 3 validaciones de compra (espacio disponible, disponibilidad parcial de boletos, presupuesto vs. precio) se aplican en el orden y lógica correctos.
 - **(10 puntos)** Las clases implementadas respetan la separación de responsabilidades MVC definida en el Diseño, con relaciones (agregación/composición) y modificadores de visibilidad correctos.

Notas:

- Para obtener una nota distinta de 0 en la parte del programa debe existir una congruencia entre el diseño y el programa. Una variación considerable (criterio subjetivo) resultará en la anulación de la parte del programa.
- Para optar a que se califique la parte del programa es necesario que el merge request lo hagan a su branch dentro del repo del curso; commits a branches equivocadas resulta en que se rechace el pull request y no se califique el código.